

Laboratorio di programmazione e Informatica 1

- A.A. 2021-2022 -

Prova di laboratorio: Primo Appello - 21 giugno 2022

ISTRUZIONI: La prima riga di ogni programma C deve contenere il proprio nome e cognome. I programmi devono essere strutturati in funzioni e completi di commenti che spieghino il procedimento. Gli esercizi vanno consegnati come file singoli denominati rispettivamente `CognomeNomeEse1.c`, `CognomeNomeEse2.c`, `CognomeNomeEse3.c`.

L'analisi, ove richiesta, dell'algoritmo utilizzato va scritta alla fine del file `.C` come commento.

L'esercizio 3 consiste nello sviluppo di una funzione da inserire poi nel file `ListeEsame1App.c` fornito a parte.

Ogni esercizio vale un max di 10 punti. Si è ammessi all'orale se si raggiunge un punteggio totale ≥ 18 con un punteggio ≥ 5 sui singoli esercizi.

ATTENZIONE! Non saranno valutati programmi che non passano la fase di compilazione. Si consiglia pertanto di "mettere sotto commento" le parti di programma che danno errore in compilazione. I file eseguibili non vanno consegnati.

Tempo a disposizione: 3 ore.

ESERCIZI

Esercizio1

In algebra lineare si definisce *Matrice di Toeplitz* una matrice in cui gli elementi di ogni diagonale (da alto-a-sinistra verso basso-a-destra) sono identici. Qui di sotto un esempio:

1	2	3	4	8
5	1	2	3	4
4	5	1	2	3
7	4	5	1	2

1. Scrivere una funzione che genera una matrice di Toeplitz a valori random nell'intervallo $[1, 10]$.
Sugg.: generare random i valori della prima riga e della prima colonna e poi assegnare tali valori al resto della matrice in modo che sia di Toeplitz.
Versione facilitata: non imporre la condizione di Toeplitz.
2. Scrivere una funzione **MaxFrequente** che calcola (e restituisce al main) il valore che compare più volte nella matrice e quante volte compare. Se tale valore non è unico se ne scelga uno qualsiasi ma la funzione dovrà segnalare che tale valore non è unico.
Versione facilitata: Non fare la segnalazione di non unicità
3. Scrivere un programma in C (funzione main) che genera una matrice di dimensione 25x15 usando la funzione sopra descritta e la stampa ben formattata sullo schermo. Poi, richiamando la funzione **MaxFrequente**, calcola il valore più frequente nella matrice e la sua frequenza. Poi stampa le informazioni calcolate.
4. Analizzare la complessità degli algoritmi utilizzati nelle funzioni.

Esercizio 2

Scrivere un programma in linguaggio C che legge dall'utente 2 stringhe di 0 e 1 di max 40 caratteri. Il programma dovrà calcolare se una delle due stringhe può essere sovrapposta sull'altra e in caso positivo la stringa sovrapposta dovrà essere cancellata dalla stringa più lunga (producendo la nuova stringa corrispondente). Tale nuova stringa sarà poi stampata sullo schermo.

Non è consentito utilizzare funzioni predefinite sulle stringhe.

Esempio: Se l'utente inserisce le stringhe:

10000100100100010000011111 e 100000111

Il programma dovrà scrivere:

La seconda stringa si sovrappone sulla prima nella posizione 15.

Cancellando la sovrapposizione, la nuova stringa e': 10000100100100011 .

Esercizio 3

Sia L una lista di interi.

1. Si scriva una funzione **EstraiSottolista** che prende in input una lista L e due interi $k1$ e $k2$ e restituisce la sottosequenza di L composta dai nodi compresi tra il primo nodo, $n1$, contenente la chiave $k1$ ed il primo nodo, $n2$, contenente la chiave $k2$. Il nodo $n2$ non deve precedere $n1$. I nodi estremi sono compresi, ovvero $n1$ ed $n2$ devono far parte della sotto-lista di output. Al termine della funzione i nodi della sottolista estratta non faranno più parte di L . Se uno dei due nodi, $n1$ o $n2$, non esiste deve essere restituita la lista iniziale non modificata.
2. Si inserisca la funzione **EstraiSottolista** nel codice **EseListeEsame1app.c** . Il main dovrà chiedere all'utente i due valori $k1$ e $k2$, eseguire **EstraiSottolista** e poi stampare in output la sottolista estratta e la lista originale modificata.
3. Si discuta la complessità degli algoritmi utilizzati in funzione della lunghezza della lista L . (inserirlo alla fine del file .C come commento).

Versione Semplificata: Non prevedere il caso in cui i nodi $n1$ o $n2$ non esistono.